

1 Euler Tour

```

1 int timer = 0;
2 // Queries on subtree range [in,out)
3 function<void(int,int)> euler_tour = [&](int u, int
    p) -> void {
4     in[u] = timer++;
5     for(int v : adj[u])
6         if( v != p )
7             euler_tour(v,u);
8     out[u] = timer;
9 };
10 euler_tour(1,0);
11 //===== Queries with Fenwick Tree
12 BIT<ll> bit(V);
13 for(int u = 1; u <= V; u++) bit.update(in[u]+1,v[u])
    ;

```

```

14 // Update 'node' to 'val' (set operation)
15 bit.update(in[node]+1,val-v[node]);
16 v[node] = val;
17 // Query (ET is 0-idx, but BIT 1-idx)
18 cout << bit.query(in[node]+1,out[node]) << endl;
19 //===== Queries with Segment Tree
20 vll flat(V+1);
21 for(int i = 1; i <= V; i++)
22     flat[ in[i] ] = v[i];
23 Segment_tree st(V);
24 st.build(flat,1,0,V-1);
25 st.update(1,in[node],0,V-1,val);
26 st.query(1,in[node],out[node]-1,0,V-1);

```

Listing 1: Euler Tour implementation, runs in $O(|V| + |E|)$. Works with an 1-indexed logic, but the time mapping is 0-indexed. Brings necessary logic to work with BIT and ST templates.